

Introduction to Algorithmic Information Theory

NICK SZABO

1996

[External link](#)

Recent discoveries have unified the fields of computer science and information theory into the field of algorithmic information theory. This field is also known by its main result, Kolmogorov complexity. Kolmogorov complexity gives us a new way to grasp the mathematics of information, which is used to describe the structures of the world. Information is used to describe the cultural structures of science, legal and market institutions, art, music, knowledge, and beliefs. Information is also used in describing the structures and processes of biological phenomena, and phenomena of the physical world. The most obvious application of information is to the engineering domains of computers and communications. This essay will provide an overview of the field; only passing knowledge of computer science and probability theory is required of the reader.

We start by describing objects with binary strings. We take the length of the shortest string description of an object as a measure of that object's complexity. Let's look at the most traditional application of information theory, communications. The sender and receiver each know specification method L . Message x can be transmitted as y , such that $L(y) = x$. This is written as $y : L(y) = x$, where the ":" is read as "such that." The cost of transmission is the length of y , $|y|$. The least cost is

$$\min\{|y| : L(y) = x\}$$

This minimal $|y|$ is the *descriptive complexity* of x under specification method L .

A universal description method should have the following properties:

- it should be independent of L , within a constant offset, so that we can compare the complexity of any object with any other object's complexity.
- the description should in principle be performable by either machines or humans

Such a method would give us a measure of absolute information content, the amount of data which needs to be transmitted in the absence of any other *a priori* knowledge.

The description method that meets these criteria is the *Kolmogorov complexity*: the size of the shortest program (in bits) that without additional data, computes the string and terminates. Formally:

$$K(x) = \min\{|p| : U(p) = x\}$$

where U is a universal Turing machine.

Variations on Kolmogorov Complexity

The *conditional* Kolmogorov complexity of string x given string y is

$$K(x | y) = \min\{|p| : U(p, y) = x\}$$

The length of the shortest program that can compute both x and y and a way to tell them apart is

$$K(x, y) = \min\{|p| : U(p) = xy\}$$

Examples of Kolmogorov Complexity

1. Pi is an infinite sequence of seemingly random digits, but it contains only a few bits of information: the size of the short program that can produce the consecutive bits of pi forever. Informally we say the descriptonal complexity of pi is a constant. Formally we say $K(\pi) = O(1)$, which means “ $K(\pi)$ does not grow.”

2. A truly random string is not significantly compressible; its description length is within a constant offset of its length. Formally we say $K(x) = \Theta(|x|)$, which means “ $K(x)$ grows as fast as the length of x .”

Algorithmic Probability and Induction

The combinations that can arise in a long series of coin flips can be divided into regular sequences, which are highly improbable, and irregular sequences, which are vastly more numerous. Wherever we see symmetry or regularity, we seek a cause. Compressibility implies causation. We can rid ourselves of the problematic nature of traditional inductive probability by redefining probability in terms of computational theory, via Kolmogorov complexity: The chance of generating string x in the form of a short program $p : U(p) = x$ is

$$P(p) = 2^{(|x| - K(x))}$$

It is $2^{(|x| - K(x))}$ times more likely that x arose as the result of an algorithmic process than by a random process.

Variations on Algorithmic Probability

If the algorithm does not generate the exact string, we can include error (called “distortion” in information theory) as part of the description of the data:

$$P(p) = 2^{(|x| - D(x, y) - K(x))}$$

The probability that a random program outputs x is called the “universal discrete semimeasure,” and is given by

$$m(x) = \sum_{p: U(p)=x} 2^{-|p|} + c$$

The algorithmic probability of x is given by

$$R(x) = 2^{-K(x)}$$

Coding theorem:

$$K(x) = \Theta(-\log m(x))$$

Conditional coding theorem:

$$K(x \mid y) + \Theta(1) = -\log m(x \mid y)$$

Prediction

Rudolf Carnap suggested that we predict by taking a weighted sum of all explanations in a given description language. Ray Solomonoff generalized this to the universal description language of Turing machines, to come up with the following universal prediction procedure: take a weighted combination of all programs that explain the data and see what they print next.

Occam's Razor

“It is vain to do with more what can be done with fewer.” Kolmogorov complexity provides an objective measure for simplicity. Computable approximations for specific combinatorial data structures have been given starting with Rissanen and Wallace. The *minimum description length* principle says to minimize the weighted sum of the model's structural complexity, and the error between the model and the data.

Distance

Distance, as the remoteness of two bodies of knowledge, was first recognized in the field of hermeneutics, the interpretation of traditional texts such as legal codes. To formalize this idea, consider two photographs represented as strings of bits. The *Hamming distance* is an unsatisfactory measure since a picture and its negative, quite similar to each other and each easily derived from the other, have a maximal Hamming distance. A more satisfactory measure is the *information distance* of Li and Vitanyi:

$$E(x, y) = \max(K(y \mid x), K(x \mid y))$$

This distance measure accounts for any kind of similarity between objects. This distance also measures the shortest program that transforms x into y and y into x . The minimal amount of irreversibility required to transform string x into string y is given by

$$KR(x, y) = K(y \mid x) + K(x \mid y)$$

Reversible computation can be used to minimize the amount of heat given off by a computing device. KR gives us the minimal amount of work required to create and destroy bits during a computation in which the intermediary program is not retained.

Superficiality and Sophistication

Charles Bennett has discovered an objective measurement for sophistication. An example of sophistication is the structure of an airplane. We couldn't just throw parts together into a vat, shake them up, and hope thereby to assemble a flying airplane. A flying structure is vastly improbable; it is far outnumbered by the wide variety of non-flying structures. The same would be true if we tried to design a flying plane by throwing a bunch of part templates down on a table and making a blueprint out of the resulting overlays.

On the other hand, an object can be considered superficial when it is not very difficult to recreate another object to perform its function. For example, a garbage pit can be created by a wide variety of random sequences of truckfuls of garbage; it doesn't matter much in which order the trucks come.

More examples of sophistication are provided by the highly evolved structures of living things, such as wings, eyes, brains, and so on. These could not have been thrown together by chance; they must be the result of an adaptive algorithm such as Darwin's algorithm of variation and selection. If we lost the genetic code for vertebrate eyes in a mass extinction, it would take nature a vast number of animal lifetimes to re-evolve them. A sophisticated structure has a high replacement cost.

Bennett calls the computational replacement cost of an object its logical depth. Loosely speaking, depth is the necessary number of steps in the causal path linking an object with its plausible origin. Formally, it is the time required by the universal Turing machine to compute an object from its compressed original description.

In some cases it makes sense to define value in terms of replacement cost. If we see an organism or tradition of great replacement cost we may on those grounds alone deem it valuable; in such cases logical depth gives us an objective measure of that value.

Part and Whole

Satisfactory compression of a large string can be computationally infeasible, while satisfactory compression of one part of that string may depend on other parts. Is it easier to compress small parts of a string by compressing the whole string at once? Or is it easier to compress small parts at once and use these results to compress the whole? How much of a string must we look at to compress a small part of that string? If we compress part of a sequence, can the pattern we have compressed it also be used to compress another part of a sequence? These are part/whole questions of a formal nature, similar to the informal but critical part/whole issues in hermeneutics. Once we have defined induction in terms of minimizing description length and distortion, the part/whole question is perhaps the sole remaining stumbling block to a consistent theory of induction free from the troubling and contradictory axioms of inductive probability.

Total and Partial Order

Our binary string representation constitutes a relation between bits known as a *total order*. A more general theory would deal with *partial orders*, but such a theory has yet to be developed. An example of where this makes a difference is where the object to be described is a series of events in the environment, and we

are observing these events with scientific observations with instruments. Sometimes the order of arrival of events is known, and sometimes it is not. With a total order we assume the order of all the events is known.

Computable description methods

$K(x)$ is in general uncomputable, since we cannot be sure a program will halt when we are testing for correctness in generating the string. The good news is that we can derive computable “entropies,” or descriptonal complexities, for computable computation structures such as polynomials, decision tress, and finite automata. These can be used as objective functions for adaptive or “learning” algorithms which construct such computing structures; for example as “fitness functions” in genetic programming. In general

$$H = \text{structural entropy} + \text{remaining sample entropy}$$

For example, when fitting a k -degree polynomial with precision d :

$$H(k, d) = kd + \Theta(\log kd) + \sum_i s \cdot (f(x_i) - y_i)^2$$

where s is some “scaling constant.”

A binary decision tree that best describes a relational database:

$$H = \# \text{ nodes in tree} + \# \text{ bits in inconsistent examples}$$

For game strategies, the number of states in finite state machine that describes the strategy is used as measure of “bounded rationality” in game theory. A more accurate measure would be the FSM’s time and space costs, plus it learnability cost, but the latter often dominates and is not well characterized by machine learning theory; the number of states is a crude approximation of the difficulty of an agent learning the strategy.

Conclusion

Algorithmic information theory is a far-reaching synthesis of computer science and information theory. Its resonances and applications go far beyond computers and communications to fields as diverse as mathematics, scientific induction and hermeneutics.

References

[Gregory Chaitin](#)

Gregory Chaitin, *Algorithmic Information Theory*

[Ming Li](#) & Paul Vitanyi, *An Introduction to Kolmogorov Complexity and Its Applications*

[Paul Vitanyi](#)

[About](#) [Contact](#) [Feed](#) [Newsletter](#) [Donate](#) [Get involved](#) [GitHub](#) [X](#) [Nostr](#)



SATOSHI NAKAMOTO INSTITUTE
ESTABLISHED BLOCK
#272,311



Satoshi Nakamoto Institute is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](#). Some works may be subject to other licenses.